Coinbase: Spend Router 2

Security Review

Cantina Managed review by:
Alireza Arjmand, Lead Security Researcher
Drastic Watermelon, Security Researcher

March 21, 2026

Contents

1 Introduction **2**
1.1 About Cantina . . . 2
1.2 Disclaimer . . . 2
1.3 Risk assessment . . . 2
1.3.1 Severity Classification . . . 2

2 Security Review Summary **3**
2.1 Scope . . . 3
2.2 Trust assumptions . . . 3
2.3 Cantina Managed Team Statement . . . 3

3 Overview **4**

4 Findings **5**
4.1 Low Risk . . . 5
4.1.1 `SpendRouter` is incompatible with fee-on-transfer tokens . . . 5
4.2 Informational . . . 7
4.2.1 Repeated validation and transfer logic across `SpendRouter` functions . . . 7
4.2.2 `SpendRouter` does not ensure the requested `value` of stETH is delivered to the recipient . . . 7
4.2.3 Code Quality and Naming Inconsistencies . . . 9
4.2.4 Strict `extraData` Length Check Limits Future Composability . . . 9
4.3 SpendPermissionManager (SPM) Issues . . . 10
4.3.1 Low Risk . . . 10
4.3.2 Informational . . . 10

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	**Impact: High**	**Impact: Medium**	**Impact: Low**
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Base is a secure, low-cost, builder-friendly Ethereum L2 built to bring the next billion users onchain.

From Mar 15th to Mar 16th the Cantina team conducted a review of spend-permissions on commit hash 0470fa98. The team identified a total of **5** in-scope issues pertaining to `SpendRouter`, and **2** additional out-of-scope issues in `SpendPermissionManager` (SPM) observed during the assessment:

SpendRouter Issues (In Scope)

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	1	0	1
Gas Optimizations	0	0	0
Informational	4	1	3
Total	**5**	**1**	**4**

SpendPermissionManager Issues (Out of Scope)

Severity	Count	Fixed	Acknowledged
Low Risk	1	0	1
Informational	1	0	1
Total	**2**	**0**	**2**

2.1 Scope

The security review had the following components in scope for spend-permissions on commit hash 0470fa98:

```
src/SpendRouter.sol
```

2.2 Trust assumptions

- `account` in a permission is assumed to be a `CoinbaseSmartWallet`. SPM's `_execute` calls `CoinbaseSmartWallet(payable(account)).execute(...)` directly. Any address can be set as `account` as long as it exposes a compatible `execute` interface, meaning a malicious account implementation could behave unexpectedly during token transfers.
- `PERMISSION_MANAGER` is assumed to be the legitimate, unmodified Coinbase SPM deployment. `SpendRouter` performs no logic validation of the SPM, only a persistent code check at deploy time.
- `MagicSpend` is assumed to correctly validate withdraw request signatures and enforce nonce uniqueness. `SpendRouter` passes `WithdrawRequest` through to SPM without any independent validation.
- The user is assumed to have reviewed both `executor` and `recipient` before signing, as neither is validated for legitimacy on-chain beyond `recipient != address(0)`.
- `Multicallable` callers are assumed to understand that `msg.value` is shared across all batched sub-calls.

2.3 Cantina Managed Team Statement

Coinbase engaged with Cantina to conduct a security review of the `SpendRouter` contract. We would like to thank the Coinbase team for their responsiveness and constructive engagement throughout the review process. No significant issues were identified during the assessment, and the protocol is expected to operate as intended.

3 Overview

`SpendRouter` is a singleton router contract developed by Coinbase that sits on top of the `SpendPermissionManager` (SPM) to enable authorized third parties to pull tokens from a user's smart account and forward them to a designated recipient in a single atomic transaction. The contract is designed to work exclusively with `CoinbaseSmartWallet` accounts and supports both native ETH (using the ERC-7528 sentinel address) and ERC-20 tokens.

The core abstraction `SpendRouter` introduces is the separation of the executing party from the token recipient. In SPM's native design, the spender and the ultimate recipient of funds are the same address. `SpendRouter` acts as the registered spender on-chain while embedding the actual intended recipient inside the permission's `extraData` field as a 64-byte ABI-encoded `(executor, recipient)` pair. The `executor` is the address authorized to trigger the spend, and the `recipient` is the address that ultimately receives the tokens. This allows protocols and off-chain operators to route user funds to arbitrary destinations while still satisfying SPM's permission model.

`SpendRouter` exposes four primary execution paths. The first, `spendAndRoute`, operates against a permission that has already been approved on-chain. The second, `spendAndRouteWithSignature`, combines on-chain approval via the user's EIP-712 signature with the spend in a single transaction. The third and fourth paths, `spendWithWithdrawAndRoute` and `spendWithWithdrawAndRouteWithSignature`, extend the above with an atomic `MagicSpend` withdrawal that pre-funds the user's account from an off-chain paymaster before the spend is executed, enabling spending of funds not yet resident on-chain. All four functions share the same execution structure: decode and validate `extraData`, verify `msg.sender == executor`, optionally approve the permission, call into SPM to pull tokens to `SpendRouter`, and finally forward them to the `recipient`.

The contract inherits from Solady's `Multicallable`, allowing multiple routing operations to be batched into a single transaction. Permission revocation is also supported through `revokeAsSpender`, which allows the authorized `executor` to permanently invalidate a permission on behalf of the spender role held by `SpendRouter`.

`SpendRouter` interacts with three external contracts. SPM manages permission state, enforces allowance periods, and executes token transfers from user accounts to `SpendRouter`. `MagicSpend` is an ERC-4337 paymaster that can fund accounts on demand via signed withdraw requests. `PublicERC6492Validator` handles signature validation for counterfactual accounts using ERC-6492, enabling permissions to be approved before an account is deployed on-chain.

4 Findings

4.1 Low Risk

4.1.1 `SpendRouter` is incompatible with fee-on-transfer tokens

Severity: Low Risk

Context: SpendRouter.sol#L101-L117, SpendRouter.sol#L129-L152, SpendRouter.sol#L165-L185, SpendRouter.sol#L198-L222

Description: All routing functions in `SpendRouter` follow the same pattern: they call `SpendPermissionManager.spend` to pull `value` tokens into the router, then forward exactly `value` tokens to the `recipient`.

`SpendPermissionManager.spend` internally calls `_transferFrom`, which transfers `value` tokens from the user's account to the `SpendRouter`.

For fee-on-transfer tokens, the actual amount received by the router is less than `value` due to the fee deducted during transfer. The router then attempts to forward the full `value` to the recipient, which will revert due to insufficient balance.

Proof of Concept: Add the following content to a `test/src/SpendRouter/FeeOnTransferForkPoCTest.t.sol` file and execute it with `forge t --mc FeeOnTransferForkPoCTest`:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import {ERC20} from "openzeppelin-contracts/contracts/token/ERC20/ERC20.sol";
import {SpendPermissionManager} from "src/SpendPermissionManager.sol";
import {SpendRouterTestBase} from "test/src/SpendRouter/SpendRouterTestBase.sol";

contract FeeOnTransferForkPoCTest is SpendRouterTestBase {
    uint256 internal constant FEE_BPS = 100; // 1%
    MockFeeOnTransferToken internal feeToken;
    address internal constant FEE_COLLECTOR = address(0xFEE);

    function setUp() public virtual override {
        bool enableFork = vm.envOr("SPEND_PERMISSIONS_ENABLE_FORK", false);
        if (enableFork) {
            string memory forkUrl = vm.envString("SPEND_PERMISSIONS_FORK_URL");
            vm.createSelectFork(forkUrl);
        }
        super.setUp();

        feeToken = new MockFeeOnTransferToken(FEE_BPS, FEE_COLLECTOR);
    }

    function _createFeePermission(uint160 allowance)
        internal
        view
        returns (SpendPermissionManager.SpendPermission memory)
    {
        uint48 start = uint48(block.timestamp);
        uint48 period = 1 days;
        uint48 end = start + period;
        return _createPermission(address(feeToken), allowance, period, start, end, 0);
    }

    function test_fork_feeOnTransfer_routerRevertsWithoutPrefund() public {
        uint160 spendAmount = 100e18;

        SpendPermissionManager.SpendPermission memory permission =
        ↪  _createFeePermission(spendAmount);
        _approvePermission(permission);

        feeToken.mint(address(account), spendAmount);
```

```
        vm.prank(executor);
        vm.expectRevert();
        router.spendAndRoute(permission, spendAmount);

        // Whole tx reverts: state remains unchanged.
        assertEq(feeToken.balanceOf(address(account)), spendAmount);
        assertEq(feeToken.balanceOf(address(router)), 0);
        assertEq(feeToken.balanceOf(recipient), 0);
        assertEq(feeToken.balanceOf(FEE_COLLECTOR), 0);
    }

    function test_fork_feeOnTransfer_doubleFeeWhenRouterPrefunded() public {
        uint160 spendAmount = 100e18;
        uint256 singleFee = feeToken.feeFor(spendAmount);

        SpendPermissionManager.SpendPermission memory permission =
        ↪  _createFeePermission(spendAmount);
        _approvePermission(permission);

        // Fund account with exactly spendAmount and pre-fund router with the expected
        ↪  first-hop shortfall.
        feeToken.mint(address(account), spendAmount);
        feeToken.mint(address(router), singleFee);

        vm.prank(executor);
        router.spendAndRoute(permission, spendAmount);

        uint256 expectedRecipient = uint256(spendAmount) - singleFee;

        // User account is debited by full spendAmount on first hop.
        assertEq(feeToken.balanceOf(address(account)), 0);

        // Recipient receives less than requested because second hop is taxed again.
        assertEq(feeToken.balanceOf(recipient), expectedRecipient);

        // Tax is collected on both hops: account->router and router->recipient.
        assertEq(feeToken.balanceOf(FEE_COLLECTOR), singleFee * 2);
    }
}

contract MockFeeOnTransferToken is ERC20 {
    uint256 public immutable feeBps;
    address public immutable feeCollector;

    constructor(uint256 _feeBps, address _feeCollector) ERC20("Fee On Transfer", "FOT") {
        feeBps = _feeBps;
        feeCollector = _feeCollector;
    }

    function mint(address to, uint256 amount) external {
        _mint(to, amount);
    }

    function feeFor(uint256 amount) public view returns (uint256) {
        return (amount * feeBps) / 10_000;
    }

    function transfer(address to, uint256 amount) public virtual override returns (bool)
    ↪  {
        _transferWithFee(_msgSender(), to, amount);
        return true;
    }

    function transferFrom(address from, address to, uint256 amount) public virtual
    ↪  override returns (bool) {
```

```
        _spendAllowance(from, _msgSender(), amount);
        _transferWithFee(from, to, amount);
        return true;
    }

    function _transferWithFee(address from, address to, uint256 amount) internal {
        uint256 fee = feeFor(amount);
        uint256 net = amount - fee;
        super._transfer(from, feeCollector, fee);
        super._transfer(from, to, net);
    }
}
```

Recommendation: Document that `SpendRouter` does not support fee-on-transfer tokens.

Alternatively if support is desired, the router should check its actual token balance change caused by the spend call and forward the received amount rather than the requested `value`.

Coinbase: Acknowledged and documented via natspec on PR 82.

Cantina Managed: Acknowledged.

4.2 Informational

4.2.1 Repeated validation and transfer logic across `SpendRouter` functions

Severity: Informational

Context: SpendRouter.sol#L101-L117, SpendRouter.sol#L129-L152, SpendRouter.sol#L165-L185, SpendRouter.sol#L198-L222

Description: The four routing functions in `SpendRouter.sol` (`spendAndRoute`, `spendAndRouteWithSignature`, `spendWithWithdrawAndRoute`, `spendWithWithdrawAndRouteWithSignature`) each independently repeat two blocks of logic:

1. Authorization and validation: decoding `extraData`, checking `msg.sender == executor`, and validating the recipient is non-zero.
2. Token forwarding: branching on native ETH vs ERC-20 for the outbound transfer.

Recommendation: Extract the repeated logic into internal helpers: `_validateExtraData` for the authorization checks, and `_transferOut` for the token forwarding branch.

Coinbase: Fixed in PR 82.

Cantina Managed: Verified the fixes.

4.2.2 `SpendRouter` does not ensure the requested `value` of stETH is delivered to the recipient

Severity: Informational

Context: SpendRouter.sol#L101-L117, SpendRouter.sol#L129-L152, SpendRouter.sol#L165-L185, SpendRouter.sol#L198-L222

Description: `SpendRouter` routes ERC-20 tokens through a two-hop transfer path: `SpendPermissionManager._transferFrom` pulls tokens from the user's account into the router, then the router forwards the same `value` to the final recipient.

stETH uses a shares-based accounting model internally. Each transfer may incur a rounding error of up to 1 or 2 wei due to the share-to-balance conversion. Since the routing path involves two `transfer/transferFrom` calls, the cumulative precision loss is up to 4 wei per routed spend.

As a result, `value` is fully debited from the user's spend allowance and emitted in the `SpendRouted` event, but the recipient receives up to 4 wei less than `value`. The discrepancy stems from the router using `transfer` and `transferFrom` rather than stETH's `transferShares`, which as noted in Lido's integration guide, enables exact-amount transfers by operating directly on the underlying shares.

The same issue applies to all four routing functions: `spendAndRoute`, `spendAndRouteWithSignature`, `spendWithWithdrawAndRoute`, and `spendWithWithdrawAndRouteWithSignature`.

Proof of Concept: Add the following content to a `test/src/SpendRouter/stETHSpendRouter.t.sol` file and execute the test with `forge t --mc StETHSpendRouterTest`:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import {console2} from "forge-std/Test.sol";
import {IERC20} from "openzeppelin-contracts/contracts/token/ERC20/IERC20.sol";
import {SpendPermissionManager} from "src/SpendPermissionManager.sol";
import {SpendRouterTestBase} from "test/src/SpendRouter/SpendRouterTestBase.sol";

contract StETHSpendRouterTest is SpendRouterTestBase {
    IERC20 constant STETH = IERC20(0xae7ab96520DE3A18E5e111B5EaAb095312D7fE84);

    /// @notice Large stETH holder to impersonate for funding.
    address constant STETH_WHALE = 0x5fAD9e2eE7aEbFFAc0a909264F85b1eEB216e3dd;

    function setUp() public override {
        vm.createSelectFork(vm.envString("ETH_RPC_URL"), 24686985);
        super.setUp();
    }

    function _approvePermissionDirect(SpendPermissionManager.SpendPermission memory
    ↪  permission) internal {
        vm.prank(permission.account);
        permissionManager.approve(permission);
    }

    function test_stETH_spendAndRoute_precisionLoss() public {
        uint160 allowance = 10 ether;
        uint48 period = 1 days;
        uint48 start = uint48(block.timestamp);
        uint48 end = start + period;

        // Create and approve a permission for stETH
        SpendPermissionManager.SpendPermission memory permission =
            _createPermission(address(STETH), allowance, period, start, end, 0);
        _approvePermissionDirect(permission);

        // Fund the account with stETH from a whale
        vm.prank(STETH_WHALE);
        STETH.transfer(address(account), uint256(allowance));

        // Rounding issue triggers on stETH transfer to account, yield 1 less wei
        assertEq(STETH.balanceOf(address(account)), allowance -1, "rounding issue did not
        ↪  trigger");

        uint160 spendAmount = 1 ether;

        // Record balances before spend
        uint256 recipientBalBefore = STETH.balanceOf(recipient);
        uint256 routerBalBefore = STETH.balanceOf(address(router));

        // Execute the spend-and-route
        vm.prank(executor);
        router.spendAndRoute(permission, spendAmount);

        // Measure actual delivery
        uint256 recipientBalAfter = STETH.balanceOf(recipient);
        uint256 routerBalAfter = STETH.balanceOf(address(router));
        uint256 received = recipientBalAfter - recipientBalBefore;

        // Recipient receives at most the spend amount (possible 0-2 wei loss)
        assertEq(received, uint256(spendAmount) - 2, "didnt lose 2 wei");
    }
}
```

Recommendation: The router currently cannot avoid this rounding since it relies on the standard ERC-20 `transfer` interface. If exact stETH delivery is a requirement, the router would need to integrate stETH's `transferShares` method for stETH-specific routing. Alternatively, document that rebasing tokens like stETH are subject to a 1-2 wei precision loss per routed spend.

Coinbase: Acknowledged and documented via natspec in PR 82.

Cantina Managed: Acknowledged.

4.2.3 Code Quality and Naming Inconsistencies

Severity: Informational

Context: SpendRouter.sol#L21, SpendRouter.sol#L139, SpendRouter.sol#L240

Description:

1. The natspec for `encodeExtraData` documents `executor` as the address that will call `spendAndRoute` or `spendAndRouteWithSignature`, omitting the other executor-gated functions (`spendWithWithdrawAndRoute`, `spendWithWithdrawAndRouteWithSignature`, and `revokeAsSpender`). Consider updating the natspec to reference all executor-gated functions.
2. The error `PermissionApprovalFailed` is misleading as it can only be triggered when the permission is already revoked. Consider renaming it to `PermissionRevoked` to accurately reflect the only condition under which it fires.
3. `SpendRouter` defines the native token sentinel as `NATIVE_TOKEN_ADDRESS` while SPM defines the same constant as `NATIVE_TOKEN`. Consider aligning the constant name with SPM's `NATIVE_TOKEN`.

Coinbase: Fixed first bullet suggestion in PR 82, will leave bullets 2 and 3 as is.

Cantina Managed: Verified the fix for the first issue. The second and third issues do not require fixing.

4.2.4 Strict `extraData` Length Check Limits Future Composability

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: `decodeExtraData` enforces `extraData.length == 64`, restricting the field to exclusively encoding the `(executor, recipient)` pair. Any protocol attempting to integrate with `SpendRouter` while embedding additional metadata in `extraData` for consumption by their own logic would be rejected with `MalformedExtraData`, precluding any extensible usage of the field.

Recommendation: Relaxing the check to revert only when `extraData.length < 64` would permit integrations to append arbitrary data beyond the first 64 bytes while `SpendRouter` continues to consume only the `(executor, recipient)` pair. With this change, however, `executor` must be set to the integrating contract itself or be subject to equivalent access control. Failure to enforce this would allow the permission to be invoked directly through `SpendRouter`, bypassing any higher-level accounting or state management the integration depends on.

Coinbase: Ack: no change. After some debate, deciding that the sanity check outweighs speculative composability given that there are other ways to cryptographically bound additional data to a spend permission (including putting a hash of other extra data in the salt) that could be implemented by integrators. Furthermore, a peripheral protocol that may want to make use of additional extraData could either also build in any necessary routing needs there, or create a new version of the SpendRouter as it is stateless and easy to migrate.

Cantina Managed: Acknowledged. This issues does not need fixing and introduced no risk to the router itself.

4.3 SpendPermissionManager (SPM) Issues

> **Disclaimer:** The following issues were identified in `SpendPermissionManager`, which was **out of scope** for this review. They are included here for completeness, as they were observed during the assessment of `SpendRouter`.

4.3.1 Low Risk

`approve` Pattern Incompatible with USDT-like Tokens if Residual Allowance Exists **Severity:** Low Risk

Context: *(No context files were provided by the reviewer)*

Description: `SpendPermissionManager._transferFrom` sets an exact allowance before pulling ERC-20 tokens from the account. Tokens such as USDT revert on `approve` if the spender's current allowance is non-zero. Under normal SPM usage this path is not reachable, however if an account had previously granted SPM a non-zero ERC-20 allowance through any means, the approval call would revert, permanently blocking spends for that token until the allowance is manually cleared to zero.

Recommendation: Use `forceApprove` from OpenZeppelin's `SafeERC20` to handle this edge case defensively.

Coinbase: Acknowledged. No change as pertains to `SpendPermissionManager`.

Cantina Managed: Acknowledged. Per the issue writeup this only happens if the user deliberately grants the `SPM` a `USDT` allowance, which requires an intentional non-standard action.

4.3.2 Informational

`SpendPermissionManager` ETH Balance Assumption is Undocumented **Severity:** Informational

Context: *(No context files were provided by the reviewer)*

Description: The native ETH path in `SpendPermissionManager._transferFrom` implicitly assumes the contract holds no ETH balance prior to execution. The `receive()` guard validates the amount received per call, but `safeTransferETH` will draw from any pre-existing balance if present. This means any ETH accidentally sent to the contract can be drained by anyone triggering a native ETH spend of the corresponding amount. This behavior is intentional given SPM is not designed to hold funds, but it is undocumented.

Recommendation: Add a natspec comment to `_transferFrom` explicitly stating that `SpendPermissionManager` is not intended to hold ETH and verifying that the functionality is indeed intended, and that the correctness of the native ETH path depends on this invariant being maintained.

Coinbase: Ack: no change as pertains to `SpendPermissionManager`.

Cantina Managed: Acknowledged.